

Agent-Based SNO (Single Node OpenShift)

Connected Installation — Detailed Homelab Guide

OpenShift Container Platform 4.20.0 | VMware ESXi | RHEL-based Bastion

Field	Value
Deployment type	Agent-Based Installer (Connected / Internet-facing)
Cluster topology	Single Node OpenShift (SNO) — 1x control-plane, 0 workers
OpenShift version	4.20.0
Base domain	ocp.example.com
Cluster name	sno
Bastion role	DNS (BIND) + Load Balancer (HAProxy) + install workstation
Prepared for	Personal homelab / Prodevans daily-task documentation

This guide expands the original installation notes into a complete, step-by-step reference for standing up an Agent-Based Single Node OpenShift cluster in a VMware ESXi homelab, including DNS, load balancing, binary setup, cluster manifests, ISO creation, boot verification, and the VMware-specific fix required for a successful install.

Table of Contents

1. Overview & Architecture
2. Prerequisites & Sizing
3. Network & Node Plan
4. Subscription & Repository Setup
5. DNS Configuration (BIND)
6. Load Balancer Configuration (HAProxy)
7. Disabling firewalld & SELinux (Lab Only)
8. Downloading OpenShift Binaries
9. Pull Secret & SSH Key Preparation
10. Cluster Configuration Files
11. Generating the Agent ISO
12. Booting the Node & Monitoring Installation
13. Critical Fix: VMware disk.EnableUUID
14. Post-Install Verification
15. Troubleshooting Reference
16. Command Cheat Sheet

1. Overview & Architecture

Single Node OpenShift (SNO) runs all control-plane and worker functions on a single node, making it ideal for homelabs, edge deployments, and learning environments with limited hardware. The **Agent-Based Installer** is used here instead of the assisted/interactive installer because it works fully offline from Red Hat's SaaS console after the ISO is generated — you only need internet access to pull the installer binaries and container images during the actual cluster bring-up (hence "Connected" installation).

1.1 Components in this Lab

Component	Role
<code>bastion.sno.ocp.example.com</code>	DNS server (BIND) + HAProxy load balancer + install workstation
<code>master1.sno.ocp.example.com</code>	Single OpenShift node acting as control-plane, worker, and initial bootstrap (rendezvous host)
<code>openshift-install</code>	CLI used to generate install manifests and the agent ISO
<code>oc / kubectl</code>	CLI clients used to interact with the running cluster
<code>oc-mirror</code>	Used for mirroring registries — included here for completeness even though this is a connected install

NOTE: In a Single Node OpenShift deployment, the same physical/virtual machine serves as control-plane, worker, and the "rendezvous" host that bootstraps the cluster — there is no separate bootstrap VM as in multi-node installs.

2. Prerequisites & Sizing

Resource	Minimum	Recommended
vCPU	8 cores	8 cores
Memory	16 Gi	16 Gi (32 Gi gives headroom for operators)
Storage	120 Gi	150 Gi
Network	1 NIC (ens160 in this lab)	Static IP, no DHCP

Software prerequisites on the bastion:

- RHEL 9.x registered with a valid Red Hat subscription (or CentOS Stream / Fedora equivalent for a pure lab)
- Internet connectivity for downloading binaries, pull secret, and pulling container images during install
- A Red Hat account to obtain the pull secret from console.redhat.com
- VMware ESXi/vCenter access with permission to edit VM advanced configuration parameters

3. Network & Node Plan

All the following addresses live on the same /16 network in this lab, which keeps DNS and HAProxy configuration simple for a single-node cluster.

Name	FQDN	IP Address	Purpose
DNS server	bastion.sno.ocp.example.com	192.168.96.101	Authoritative DNS for the cluster domain
Gateway	-	192.168.10.1	Default route for cluster nodes
api	api.sno.ocp.example.com	192.168.96.101	OpenShift API endpoint (fronted by HAProxy)
api-int	api-int.sno.ocp.example.com	192.168.96.101	Internal API endpoint
*.apps	*.apps.sno.ocp.example.com	192.168.96.101	Wildcard for all routes/ingress
master1	master1.sno.ocp.example.com	192.168.96.102	The single control-plane/worker node

NOTE: Notice that api, api-int, and the wildcard *.apps all resolve to the bastion's IP (192.168.96.101), not master1. This is because HAProxy on the bastion listens on those addresses and forwards traffic to master1 on the appropriate ports (6443, 22623, 80, 443).

4. Subscription & Repository Setup

Register the bastion host so package repositories (BIND, HAProxy, etc.) are available.

```
[root@bastion ~]# subscription-manager register --username <username> \
--password <passwd> --auto-attach
[root@bastion ~]# subscription-manager list
```

NOTE: If you are using a non-RHEL base (e.g. Rocky/AlmaLinux/CentOS Stream) for the bastion only, skip subscription-manager and simply ensure the base OS repos are enabled — the bastion itself does not need to be RHEL; only the OpenShift node does.

5. DNS Configuration (BIND)

OpenShift is heavily DNS-dependent: the installer and every node must be able to resolve api, api-int, the wildcard apps domain, and each node's own hostname (forward and reverse). We use BIND on the bastion as the authoritative resolver for the lab domain.

5.1 Install BIND

```
[root@bastion ~]# yum install bind bind-utils -y
```

5.2 Configure /etc/named.conf

```
options {
    listen-on port 53 { 127.0.0.1; 192.168.96.101; };
```

```

listen-on-v6 port 53 { ::1; };
directory "/var/named";
dump-file "/var/named/data/cache_dump.db";
statistics-file "/var/named/data/named_stats.txt";
memstatistics-file "/var/named/data/named_mem_stats.txt";
secroots-file "/var/named/data/named.secreots";
recursing-file "/var/named/data/named.recursing";
allow-query { localhost; 192.168.0.0/16; any; };
};

zone "sno.ocp.example.com" IN {
    type master;
    file "forward.zone";
    allow-update { none; };
};

zone "96.168.192.in-addr.arpa" IN {
    type master;
    file "reverse.zone";
    allow-update { none; };
};

```

NOTE: Replace sno.ocp.example.com and the 96.168.192.in-addr.arpa reverse zone with values matching your own base domain and subnet before applying this to your own homelab.

5.3 Create the Forward Zone

```

[root@bastion ~]# vim /var/named/forward.zone

$TTL 1D
@   IN SOA  bastion.sno.ocp.example.com. root.sno.ocp.example.com. (
                                0      ; serial
                                1D     ; refresh
                                1H     ; retry
                                1W     ; expire
                                3H )   ; minimum
    IN NS   bastion.sno.ocp.example.com.

bastion    IN A    192.168.96.101
api        IN A    192.168.96.101
api-int    IN A    192.168.96.101
*.apps     IN A    192.168.96.101
master1    IN A    192.168.96.102

```

5.4 Create the Reverse Zone

```

[root@bastion ~]# vim /var/named/reverse.zone

$TTL 1D
@   IN SOA  bastion.sno.ocp.example.com. root.sno.ocp.example.com. (
                                0      ; serial
                                1D     ; refresh
                                1H     ; retry
                                1W     ; expire
                                3H )   ; minimum
    IN NS   bastion.sno.ocp.example.com.
    IN NS   sno.ocp.example.com.

101 IN PTR  bastion.sno.ocp.example.com.

```

```
101 IN PTR api.sno.ocp.example.com.
101 IN PTR api-int.sno.ocp.example.com.
102 IN PTR master1.sno.ocp.example.com.
```

5.5 Fix Ownership

```
[root@bastion ~]# chown root:named /var/named/reverse.zone
[root@bastion ~]# chown root:named /var/named/forward.zone
```

5.6 Validate Configuration

```
[root@bastion ~]# named-checkconf -z /etc/named.conf
zone sno.ocp.example.com/IN: loaded serial 0
zone 96.168.192.in-addr.arpa/IN: loaded serial 0
```

```
[root@bastion ~]# named-checkzone forward /var/named/forward.zone
zone forward/IN: loaded serial 0
OK
```

```
[root@bastion ~]# named-checkzone reverse /var/named/reverse.zone
zone reverse/IN: loaded serial 0
OK
```

5.7 Start & Enable Named

```
[root@bastion ~]# systemctl start named
[root@bastion ~]# systemctl enable named
[root@bastion ~]# systemctl status named
```

5.8 Verify Resolution

Run both forward and reverse lookups before moving on — a broken DNS record here is one of the most common causes of a stuck Agent-Based install.

```
[root@bastion ~]# dig -x 192.168.96.101
;; ANSWER SECTION:
101.96.168.192.in-addr.arpa. 86400 IN PTR bastion.sno.ocp.example.com.
101.96.168.192.in-addr.arpa. 86400 IN PTR api-int.sno.ocp.example.com.
101.96.168.192.in-addr.arpa. 86400 IN PTR api.sno.ocp.example.com.
```

```
[root@bastion ~]# nslookup api.sno.ocp.example.com.
Server:      192.168.96.101
Name:        api.sno.ocp.example.com
Address:     192.168.96.101
```

```
[root@bastion ~]# nslookup master1.sno.ocp.example.com.
Server:      192.168.96.101
Name:        master1.sno.ocp.example.com
Address:     192.168.96.102
```

6. Load Balancer Configuration (HAProxy)

Even with a single node, OpenShift's installer expects the API (6443), machine-config server (22623), and ingress (80/443) endpoints to be reachable through a load balancer front-end. HAProxy on the bastion fills that role and forwards every request to master1.

6.1 Install HAProxy

```
[root@bastion ~]# yum install haproxy -y
```

6.2 Edit /etc/haproxy/haproxy.cfg

```
global
    log 127.0.0.1 local2
    pidfile /var/run/haproxy.pid
    maxconn 4000
    daemon

defaults
    mode http
    log global
    option dontlognull
    option http-server-close
    option redispatch
    retries 3
    timeout http-request 10s
    timeout queue 1m
    timeout connect 10s
    timeout client 1m
    timeout server 1m
    timeout http-keep-alive 10s
    timeout check 10s
    maxconn 3000

listen api-server-6443
    bind *:6443
    mode tcp
    option httpchk GET /readyz HTTP/1.0
    option log-health-checks
    balance roundrobin
    server master1 master1.sno.ocp.example.com:6443 weight 1 verify none \
        check check-ssl inter 10s fall 2 rise 3

listen machine-config-server-22623
    bind *:22623
    mode tcp
    server master1 master1.sno.ocp.example.com:22623 check inter 1s

listen ingress-router-443
    bind *:443
    mode tcp
    balance source
    server master1 master1.sno.ocp.example.com:443 check inter 1s

listen ingress-router-80
    bind *:80
    mode tcp
    balance source
    server master1 master1.sno.ocp.example.com:80 check inter 1s
```

6.3 Start & Enable HAProxy

```
[root@bastion ~]# systemctl enable --now haproxy
[root@bastion ~]# systemctl status haproxy
```

NOTE: The four listen blocks map directly to the four things a client needs from the cluster: the Kubernetes API (6443), the Machine Config Server used during node provisioning (22623), and HTTPS/HTTP ingress for routes (443/80). All four forward to the same master1 node because this is SNO — in a multi-node cluster these would load-balance across several control-plane or router-hosting nodes.

7. Disabling firewalld & SELinux (Lab Only)

```
[root@bastion ~]# systemctl disable firewalld
[root@bastion ~]# systemctl stop firewalld
[root@bastion ~]# setenforce 0
[root@bastion ~]# getenforce
Permissive
[root@bastion ~]# systemctl status firewalld
```

IMPORTANT: Disabling firewalld and setting SELinux to Permissive simplifies bring-up in a closed homelab, but this is not a production practice. For anything beyond a personal lab, instead open only the required ports (53, 80, 443, 6443, 22623) in firewalld and keep SELinux Enforcing.

8. Downloading OpenShift Binaries

Download the installer, client, and oc-mirror tool for the exact OpenShift version you plan to deploy (4.20.0 in this guide), and confirm all three report the same version before proceeding.

```
[root@bastion ~]# mkdir ocp-binary && cd ocp-binary

[root@bastion ocp-binary]# wget https://mirror.openshift.com/pub/openshift-v4/clients/ocp/4.20.0/openshift-install-linux.tar.gz
[root@bastion ocp-binary]# wget https://mirror.openshift.com/pub/openshift-v4/clients/ocp/4.20.0/openshift-client-linux.tar.gz
[root@bastion ocp-binary]# wget https://mirror.openshift.com/pub/openshift-v4/clients/ocp/4.20.0/oc-mirror.tar.gz

[root@bastion ocp-binary]# ls -lrth
-rw-r--r--. 1 root root 124M oc-mirror.tar.gz
-rw-r--r--. 1 root root 74M openshift-client-linux.tar.gz
-rw-r--r--. 1 root root 395M openshift-install-linux.tar.gz

[root@bastion ocp-binary]# tar xvf openshift-install-linux.tar.gz -C /usr/local/bin/
[root@bastion ocp-binary]# tar xvf openshift-client-linux.tar.gz -C /usr/local/bin/
[root@bastion ocp-binary]# tar xvf oc-mirror.tar.gz -C /usr/local/bin/
```

Verify versions match:

```
[root@bastion ocp-binary]# oc version
Client Version: 4.20.0
Kustomize Version: v5.6.0
```

```
[root@bastion ocp-binary]# openshift-install version
openshift-install 4.20.0
built from commit 98df3532f59ede67f5ebb2e9ee0be2e7e89eea4b
release image quay.io/openshift-release-dev/ocp-release@sha256:d1dc76522d1e...
release architecture amd64
```



```
[root@bastion ocp-binary]# oc-mirror --v2 version
Client Version: version.Info{... GitVersion:"4.20.0-202509230043...", ...}
```

NOTE: A version mismatch between openshift-install and oc is one of the more common sources of confusing errors later in the install — always confirm all three tools line up before generating the ISO.

9. Pull Secret & SSH Key Preparation

The pull secret authorizes the node to pull OpenShift's release container images; the SSH key lets you log in to master1 for troubleshooting after boot.

```
[root@bastion ~]# mkdir secret && vim secret/pull-secret.txt
# Paste the pull secret copied from https://console.redhat.com/openshift/downloads

[root@bastion ~]# cat secret/pull-secret.txt | jq . > secret/pull-secret.json

# Generate an SSH key pair on the bastion if one does not already exist
[root@bastion ~]# ssh-keygen -t ed25519 -N '' -f ~/.ssh/id_rsa
[root@bastion ~]# cat ~/.ssh/id_rsa.pub
```

10. Cluster Configuration Files

Two YAML manifests drive an Agent-Based install: agent-config.yaml (per-host network and rendezvous details) and install-config.yaml (cluster-wide settings). Both are consumed once by openshift-install agent create image and are not needed again afterward.

10.1 agent-config.yaml

```
[root@bastion ~]# mkdir cluster-deploy && cd cluster-deploy
[root@bastion cluster-deploy]# vim agent-config.yaml

apiVersion: v1alpha1
kind: AgentConfig
metadata:
  name: sno-agent-config
rendezvousIP: 192.168.96.102      # the node that bootstraps the cluster
hosts:
  - hostname: master1.sno.example.com
    role: master
    interfaces:
      - name: ens160              # from: nmcli connection show
        macAddress: 00:0c:29:7b:67:73 # the VM's NIC MAC address
    networkConfig:
      interfaces:
        - name: ens160
          type: ethernet
          state: up
          ipv4:
            enabled: true
            dhcp: false
            address:
              - ip: 192.168.96.102
                prefix-length: 16      # matches the lab subnet mask
          ipv6:
            enabled: false
```

```

dns-resolver:
  config:
    server:
      - 192.168.96.101          # bastion DNS server
routes:
  config:
    - destination: 0.0.0.0/0
      next-hop-address: 192.168.10.1  # gateway
      next-hop-interface: ens160

```

NOTE: rendezvousIP must point at the node that will act as the temporary bootstrap host. In SNO this is simply the one and only node — master1.

10.2 install-config.yaml

```
[root@bastion cluster-deploy]# vim install-config.yaml
```

```

apiVersion: v1
baseDomain: ocp.example.com
compute:
  - hyperthreading: Enabled
    name: worker
    replicas: 0
controlPlane:
  hyperthreading: Enabled
  name: master
  replicas: 1
metadata:
  name: sno
networking:
  clusterNetwork:
    - cidr: 10.128.0.0/14
      hostPrefix: 23
  networkType: OVNKubernetes
  serviceNetwork:
    - 172.30.0.0/16
  machineNetwork:
    - cidr: 192.168.96.0/16          # master node's network range
platform:
  none: {}
fips: false
pullSecret: '<paste contents of pull-secret.json here>'
sshKey: '<paste contents of ~/.ssh/id_rsa.pub here>'

```

NOTE: replicas: 0 for compute and replicas: 1 for controlPlane is what tells the installer this is a Single Node OpenShift deployment rather than a standard multi-node cluster.

11. Generating the Agent ISO

Install the nmstate package (required by the agent installer for the declarative network configuration in agent-config.yaml) before generating the ISO.

```
[root@bastion ~]# yum install nmstate -y
```

```
[root@bastion cluster-deploy]# openshift-install agent create image
```

```
[root@bastion cluster-deploy]# ll
-rw-r--r--. 1 root root 1326159872 agent.x86_64.iso
drwxr-x---. 2 root root          50 auth/
-rw-r--r--. 1 root root          14 rendezvousIP
```

IMPORTANT: openshift-install consumes and deletes/rewrites install-config.yaml and agent-config.yaml during this step — keep copies elsewhere if you'll need to regenerate the ISO later.

12. Booting the Node & Monitoring Installation

Attach agent.x86_64.iso as the boot CD/DVD for the master1 VM in VMware (vSphere Client → Edit Settings → CD/DVD Drive → Datastore ISO File), power it on, and boot from the ISO.

During boot you'll see CoreOS start up, apply its Ignition configuration, and identify itself as the rendezvous host:

```
Red Hat Enterprise Linux CoreOS 9.6.20250826-1 (Plow)
OpenShift Agent Installer image for sno.ocp.example.com
ens160: 192.168.96.102
Ignition: user-provided config was applied
This host (192.168.96.102) is the rendezvous host.
```

```
Waiting for services:
[failed] failed Service that starts cluster installation
```

```
Cluster installation in progress
master1 login:
```

12.1 Watch installation logs

```
[root@master1 ~]# journalctl -f
```

```
Hosts known and ready for cluster installation (0/1)
```

NOTE: 0/1 Ready is expected and temporary while cluster services are initializing. Only treat it as a problem if it persists alongside an explicit validation failure message in the journal, as covered in the next section.

13. Critical Fix: VMware disk.EnableUUID

On VMware, the install commonly stalls with the node stuck at 0/1 Ready, and the journal reports exactly one validation failure:

```
Your installation is failing for ONLY this validation:
VSphere disk.EnableUUID isn't enabled for this virtual machine
```

OpenShift's vSphere/VMware CSI and node identity code relies on a stable disk UUID to correctly track the VM's storage, which VMware only exposes when disk.EnableUUID is explicitly set to TRUE on the VM's advanced configuration.

13.1 Fix via vSphere UI

- 1 Power OFF the master1 VM.
- 2 Right-click the VM → **Edit Settings**.
- 3 Go to **VM Options** → **Advanced** → **Configuration Parameters**.
- 4 Click **Add** and set: `disk.EnableUUID = TRUE`
- 5 Click **Save**.
- 6 Power the VM back ON.

13.2 Fix via govc (CLI alternative)

If you manage the lab with govc or PowerCLI instead of the UI, the same parameter can be set from the command line — useful if you rebuild VMs often:

```
# govc example
govc vm.change -vm master1 -e disk.enableUUID=TRUE
```

```
# PowerCLI example
Get-VM master1 | New-AdvancedSetting -Name "disk.EnableUUID" -Value "TRUE" -Confirm:$false
```

13.3 Confirm the Node Recovers After Reboot

```
Red Hat Enterprise Linux CoreOS 9.6.20250826-1 (Plow)
ens160: 192.168.96.102
Ignition: ran on 2026-05-17 09:56:12 UTC (this boot)
Ignition: user-provided config was applied
This host (192.168.96.102) is the rendezvous host.
```

```
Waiting for services:
[failed] failed Service that starts cluster installation
```

```
Cluster installation in progress
master1 login:
```

```
[root@master1 ~]# systemctl status kubelet
● kubelet.service - Kubernetes Kubelet
  Active: active (running) since ...
  Main PID: 26222 (kubelet)

[root@master1 ~]# systemctl status crioc
● crioc.service - Container Runtime Interface for OCI (CRI-O)
  Active: active (running) since ...
```

NOTE: Set `disk.EnableUUID = TRUE` on every VMware VM you plan to use as an OpenShift node — this is not SNO-specific, and the same validation failure appears on multi-node vSphere clusters too.

14. Post-Install Verification

Once kubelet and crioc are healthy on master1, verify the cluster from the bastion using the generated kubeconfig.

```
[root@bastion ~]# export KUBECONFIG=/root/cluster-deploy/auth/kubeconfig
[root@bastion ~]# oc whoami
system:admin
```

```
[root@bastion ~]# oc get no
```

NAME	STATUS	ROLES	AGE	VERSION
master1.sno.example.com	Ready	control-plane, master, worker	29h	v1.33.5

```
[root@bastion ~]# oc get co
```

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED	SINCE
authentication	4.20.0	True	False	False	6h27m
baremetal	4.20.0	True	False	False	29h
cloud-controller-manager	4.20.0	True	False	False	29h
cloud-credential	4.20.0	True	False	False	29h

```
[root@bastion ~]# oc get mcp
```

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
master	rendered-master-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	True	False	False

14.1 Recommended follow-up checks

- oc get co — every ClusterOperator should reach AVAILABLE=True, DEGRADED=False
- oc get mcp — MachineConfigPool should show UPDATED=True, UPDATING=False
- oc get clusterversion — confirms the cluster has fully reconciled to 4.20.0
- oc get pods -A | grep -v Running — flags anything stuck outside a healthy state
- Browse the console at <https://console-openshift-console.apps.sno.ocp.example.com>

15. Troubleshooting Reference

Symptom	Likely Cause	Fix
Node stuck at 0/1 Ready with a disk.EnableUUID validation message	VMware VM Advanced Config missing the parameter	Set disk.EnableUUID = TRUE (Section 13), then reboot
nslookup/dig fails for api or *.apps	Forward/reverse zone file typo or named not restarted	Re-run named-checkzone, systemctl restart named
HAProxy shows server DOWN for master1	master1 not yet listening on 6443/22623/443/80, or firewall blocking	Wait for boot to progress further; check firewall status on both hosts
openshift-install / oc / oc-mirror versions differ	Binaries downloaded from mismatched release paths	Re-download all three from the same /clients/ocp// path
Pull secret rejected during image pull	Pull secret not converted to valid JSON, or expired	Re-copy from console.redhat.com and re-run cat pull-secret.txt jq .
Cluster install hangs indefinitely with no validation errors	DNS resolution asymmetry between forward and reverse records	Verify every forward record has a matching PTR and vice versa

16. Command Cheat Sheet

```
# DNS
systemctl restart named
named-checkconf -z /etc/named.conf
dig -x <ip> ; nslookup <fqdn>
```

```
# HAProxy
systemctl restart haproxy
echo "show stat" | socat stdio /var/run/haproxy.pid # if stats socket enabled

# Install
openshift-install agent create image --dir cluster-deploy
openshift-install agent wait-for install-complete --dir cluster-deploy

# Cluster
export KUBECONFIG=/root/cluster-deploy/auth/kubeconfig
oc whoami
oc get no
oc get co
oc get mcp
oc get clusterversion

# Node-level (SSH to master1)
journalctl -f
systemctl status kubelet
systemctl status crio
```

Prepared as a personal homelab / Prodevans training reference — based on hands-on Agent-Based SNO installation notes, expanded with context, verification steps, and a troubleshooting quick-reference.